

CITATION INTELLIGENCE AGENT

Citelens - Citation Cleaner

From a topic, an author, or a paper title — to the references that actually matter, ranked by how often the field cites them, with every citing passage traced back to its source.

Who, when, and what for

Before the machinery, the use case. Citelens is built for the moment when you are staring at an unfamiliar research area and need to know which works the literature keeps coming back to — not just a flat list of references, but a frequency-ranked, evidence-backed map of the conversation.

Who it's for

- Researchers entering a new field
- Grad students writing a lit review
- R&D / policy landscape scans
- Librarians & research support

When to use it

- You have a topic, not a ref list
- You want the most-cited works first
- You need to know where it's cited
- You're cleaning refs at scale

The headline feature: citation frequency, with receipts

Give the agent a starting signal — an author, a piece of terminology, or a paper title — and it retrieves the most relevant papers from OpenAlex, say the top 3. It then opens each of those papers and reads who they cite. Every reference is tallied across the set and ranked from most to least cited.

The payoff is a report that reads like a field guide. For each top co-cited work, the report shows not just how many papers cite it, but exactly where: the page number, the citation marker, and the surrounding sentence.

Example output

Real example — query: diffusion models (source: *Diffusion Models Beat GANs on Image Synthesis, 2021, cited 2171×*)

→ 73 references extracted · 135 citances · 71 canonical works after deduplication

Analyzing and Improving the Image Quality of StyleGAN (2019) — Karras; Laine et al. — appears 6 times in text

p. 1 · [5, 28, 51] · "infinite high-quality synthetic images [5, 28, 51] and highly diverse human speech..."

p. 4 · [27, 28, 25] · "some papers [27, 28, 25] compare against arbitrary subsets..."

p. 10 · [28] · "StyleGAN2 [28] 3.84"

... and 3 more citing passages.

The challenge: why this task is genuinely hard

Ranking citations sounds like counting. It isn't — and that gap is the whole reason this is an agent rather than a script.

1. The same work wears a dozen disguises

A single paper shows up across a corpus as reordered author names, initials versus full names, drifting title casing, typos, missing venues, and edition variants. Count naively and you split one work into five. The system's first job is to collapse every messy variant into one canonical entry per unique work before it counts anything — otherwise the frequency ranking is meaningless.

2. Look-alike titles are not the same work

The mirror-image failure: two genuinely different books are both titled Generalized Additive Models. Merge them and you fabricate a "highly cited" work that never existed. Telling these apart often requires reading the context in which each was cited — exactly the kind of judgment a regex cannot make.

3. The hard cases aren't catchable by rules

The difficulty comes in layers, and only the top layer is tractable with deterministic tooling:

Layer	Example & tractability
Format noise	Extra spaces, page suffixes, & versus and → regex handles this
Field order	Smith, J. versus J. Smith → partly tractable
Abbreviation	John R. Smith versus J. R. Smith → requires judgment
Casing & typos	asymptotic statistics versus misspelled variants → requires judgment
Entity ambiguity	Two different books titled Generalized Additive Models → requires reading context
Cross-corpus context	Same raw reference used differently across citing papers → requires judgment

4. The input itself is messy

Real corpora don't arrive as clean reference tables — they arrive as PDFs. Locating the reference section, splitting it into entries, and linking each in-text marker back to its source (so you can say "cited on page 4") is brittle, template-dependent work that no single rule covers.

Why an agent + LLM, not a script

Put together, these are not formatting problems — they are judgment problems. Deciding whether two strings name the same work requires reading meaning, not matching patterns. That is where the LLM earns its place. But model calls are expensive, so the architecture is deliberately frugal.

LLM frugality — measured

In a real run on a single-paper corpus (Diffusion Models Beat GANs), Stage 5 auto-resolved **62 of 66 clusters** without any model call. The LLM was invoked for only **4 genuinely ambiguous cases**.

Layer	What does the work
Format noise, field reordering	Deterministic regex / rules
Abbreviation, casing, typos	Title embeddings + clustering (qwen/qwen3-embedding-4b via OpenRouter)
Same / different judgment	LLM judge — only on uncertain clusters (deepseek/deepseek-v3 via OpenRouter)
Unresolvable clusters	Agent loop → Crossref + OpenAlex lookups

The model is the scalpel, not the hammer. Both LLM calls (structured extraction in Stage 2, same/different judgment in Stage 5) and title embeddings (Stage 4) run through OpenRouter — one API key, swappable models.

How the agent is built: pipeline architecture

Nine stages turn a query (or a folder of PDFs, or a raw reference CSV) into a ranked, traceable citation report. Color marks the kind of work each stage does — and shows at a glance how sparingly the LLM is used.

Stage	What happens
Input	Topic / author / PDF upload / raw reference CSV
Retrieval	Search OpenAlex; download open-access PDFs; keep top N papers
Stage 0	PDF parse — extract page text (PyMuPDF), split reference section, scan in-text citation markers, link each marker to its reference entry, write ParsedDocument [PDF mode only]
Stage 1	Pre-clean — regex normalisation, drop noise
Stage 2	Extract — LLM (OpenRouter) → typed structured records, batched with 8 workers
Stage 3	Block — group by surname × year
Stage 4	Embed — title embeddings (OpenRouter) + DBSCAN clustering per block
Stage 5	Judge — LLM same/different verdict on ambiguous clusters only; deterministic auto-resolve for clear cases
Stage 6	Resolve — tool-using agent loop, calls Crossref + OpenAlex to break remaining ties [uncertain clusters only]
Stage 7	Co-citation — cross-paper aggregation; tally co_cited count and in_text_hits per reference
Enrich	Fetch global citation counts for top 30 references from OpenAlex
Report	Ranked co-citation table + in-context appearances appendix (page · marker · sentence)

Two entry points, one pipeline

CSV mode starts at Stage 1 with a hand-curated reference table. No PDF download or Stage 0 needed.

PDF mode adds Stage 0 and the Retrieval step. Stage 0 also produces the in-text citances

that power the in-context appearances appendix in the report.

Two counts in the report: Co-cited (how many source papers include a reference in their bibliography) and In-text hits (how many times it appears in body text). They often diverge — a paper cited once per source paper but heavily discussed in each will have high in-text hits but moderate co-cited count.

Stage 0: the PDF ingestion channel

Stage 0 is the part worth noticing. It makes the system usable when your starting point is a folder of papers rather than a tidy CSV — and it is where the in-context citing passages that power the headline report are born.

1. Extract page text from each PDF with PyMuPDF.
2. Locate the reference section by heading and position signals.
3. Split that section into individual raw reference strings.
4. Score extraction quality by reference count, year coverage, and author-like starts.
5. Optionally call an LLM fallback (via OpenRouter) when reference-section extraction looks weak.
6. Extract header metadata: title, DOI, arXiv id, year.
7. Detect numbered and author-year in-text citation markers.
8. Link each marker back to its reference entry — the basis for "cited on page N".
9. Write a ParsedDocument with references, citances, metadata, and quality signals.

Not a black box

Every PDF produces inspectable artifacts. A bad section cut or an unlinked citation is visible before it can poison the canonical reference table or skew the frequency ranking.

Boundary

The PDF channel is wired for publisher PDFs with embedded, selectable text. It is not an OCR system for scanned documents, and it does not promise full layout reconstruction for every journal template. Low-quality parses are surfaced through quality reports and review artifacts — never silently waved through as success.

Repository layout

```
citation_cleaner/
├── schemas/      reference.py, citance.py, document.py
├── parsers/      heuristic.py, reference_section.py, citance_linker.py,
│                llm_fallback.py, quality.py
├── preclean/     rules.py
├── extractors/   llm.py, dry_run.py, cache.py
├── blocking/     surname.py
├── embedding/    providers.py, clustering.py
├── resolvers/    judge.py, agent.py, tools.py
├── pipelines/    stages.py, pdf_to_canonical.py, refs_to_canonical.py,
│                discover_pipeline.py
├── aggregate/    cocitation.py
├── discovery/    openalex.py, topic_from_paper.py
└── llm/          client.py, anthropic.py, openrouter.py, json_parse.py
```

```

scripts/      run_pipeline.py, run_discover.py, parse_pdfs.py,
              web_app.py, check_api.py, eval_stage0.py, eval_score.py
demo/        raw_refs_sample.csv, citances_sample.jsonl, sample_pdfs/
references/   schema.md, pdf_parsing.md, prompts.md, tools.md, eval.md
tests/       test_smoke.py, test_parsers.py, test_citance_linker.py,
              test_pipelines.py, test_v4_features.py (25 tests)
SKILL.md     Agent-facing contract (MCP skill descriptor)

```

The package is self-contained: SKILL.md is the agent-facing contract, citation_cleaner/ is the importable implementation, and scripts/, references/, and tests/ are bundled resources.

How to use

You can run Citelens from the command line or try it instantly in the [Playground](#) — no setup needed. For CLI use, run `--dry-run` for offline smoke tests first, and omit it only after `scripts/check_api.py` passes with a valid OpenRouter key.

Discovery workflow (recommended entry point for v4)

```

# By topic
python scripts/run_discover.py --query "diffusion models" --workdir ./run_dm
python scripts/run_discover.py --query "diffusion models" --workdir ./run_dm --sort recency

# Upload a paper — topic extracted automatically
python scripts/run_discover.py --upload mypaper.pdf --workdir ./run_mp --sort citations

# By author (fixed sort: citations)
python scripts/run_discover.py --author "Yann LeCun" --workdir ./run_lecun --n 5
python scripts/run_discover.py --author "A2208157607" --workdir ./run_lecun --n 5

```

Raw reference-table workflow

```
python scripts/run_pipeline.py --refs demo/raw_refs_sample.csv --workdir run_refs --dry-run
```

PDF corpus workflow

```
python scripts/run_pipeline.py --pdfs ./papers --workdir run_pdfs --dry-run
```

Web UI

```
python scripts/web_app.py
```

Three tabs: By topic / Upload paper / By author. OpenRouter API key is saved in localStorage.

Evaluation

Stages 1–6 are scored with pair-level F1 and cluster purity against a hand-labeled reference set. Stage 0 earns its own PDF-level metrics: reference recall, reference precision, citance recall, citance-link accuracy, fallback rate, and whether the LLM fallback actually improved extraction quality.

```

python scripts/eval_score.py --workdir ./run_2026 --eval-set eval/eval_set.csv
python scripts/eval_stage0.py --pdfs eval/pdf_eval_set --manifest eval/pdf_eval_set/manifest.csv

```